

TABLE I
SHARED NOTATION FOR SUPPLE, OFR, AND OFRSUPPLE.

Symbol	Meaning
D	Secret payload array; OFR also routes membership words as payloads.
M	Secret membership-word array; bit j of $M[i]$ indicates membership in sample j .
T, T'	Secret routing tags and their normalized copy. Control generation mutates the copy; internal writers call their supplied buffer T .
P, H	Secret length-10 one-hot tag-pair mask and tag-pair count array, respectively; both use the category order in Algorithm 15.
C	Secret control array: one-bit selection flags in Supple, two-bit OFORK words in OFR.
$C_{\text{swo}}, C_{\text{ofr}}$	The selection-bit and OFR-word arrays used together in OFRSupple; swo retains the original selection-control notation.
ℓ_c, ℓ_g	Public control-element count and current-layer OFORK gate count, respectively; $\ell_g = \lfloor n/2 \rfloor$.
$\ell_{c,\text{swo}}, \ell_{c,\text{ofr}}$	Public counts of selection bits and OFR two-bit words, respectively.
$p, p_{\text{swo}}, p_{\text{ofr}}$	Public next read/write position, measured in elements of the associated control array.
R	Secret remaining-selection-count array in SUPPLEMARK.
$S, \mathcal{S}$	A secret sample array and an ordered list of sample arrays, respectively.
F, F'	Public ordered frontier arrays; $F = \perp$ requests the default split. F' is the explicit frontier used for traversal.
n, m, k	Public current input length, sample length, and number of samples in the current task.
n_l, n_r	Public left/right output capacities, with $n_l + n_r = n$.
a, s	Public base and positive stride of an OFR logical view: wire j resides at $T[a + js]$ offline or $D[a + js]$ online.
$n^{\text{top}}, n^{\text{bot}}$	Public OFR subnetwork lengths $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$.
$n_l^{\text{top}}, n_r^{\text{top}}, n_l^{\text{bot}}, n_r^{\text{bot}}$	Public left/right output capacities of the top/bottom subnetworks.
i, j, v	Public loop indices; v indexes frontier nodes, while v_l, v_r below are secret counts.
$(s_v, k_v), n_v$	Frontier-node descriptor (starting sample-bit index, sample count) and public capacity $n_v = \min(n, mk_v)$.
μ	Public membership bit mask for the corresponding node or sample group.
ρ	Fresh secret uniform random value in $[0, 1)$ used by SUPPLEMARK.
x, y, t, c	Secret scalar input tags/items, routing tag $t = (t_l, t_r)$, and two-bit OFORK control word c .
w_l, w_r	Secret actual left/right tag-responsibility counts; bounded by capacity before normalization and equal to it afterward.
δ_l, δ_r	Secret normalization deficits $n_l - w_l$ and $n_r - w_r$.
e_l, e_r	Secret responsibility bits of the last input bypassing to top when n is odd; zero when n is even.
$n_l^{\text{top}'}, n_r^{\text{top}'}$	Secret paired-input targets $n_l^{\text{top}} - e_l$ and $n_r^{\text{top}} - e_r$; these are not array lengths.
d	Secret count of $\{10, 01\}$ pairs, i.e., $H[5]$.
$v_l, v_r; f_l, f_r$	Secret variable-pair counts and fixed top-responsibility counts for each output direction.
q_d, q_l, q_r	Secret allocation quotas; $q_d^{\min}, q_d^{\max}$ bound the feasible diagonal-pair quota.
$r_0; r_d, r_l, r_r$	Secret counts of preceding zero tags or preceding pairs in the corresponding allocation category.
u, u_*	Secret selection bits: sampling in SUPPLEMARK, tag assignment in OFRNORMALIZE, pair allocation in OFRBALANCE, or control selection in OFORKCTR.
z, z_*	Secret predicate bits: zero-tag test in OFRNORMALIZE, fixed-responsibility tests in OFRBALANCE, and routing/copy tests in OFORKCTR.
00, 10, 01, 11	Routing tags: no, left-only, right-only, and both-direction responsibility. Array components $T[i][0], T[i][1]$ refer to left/right.
00, 01, 10, 11	Control words 00, 01, 10, 11: OFORK(x, y, c) yields $(x, x), (x, y), (y, x), (y, y)$, respectively.

Indices are zero-based; $A[a : b)$ is a half-open slice (an explicit view when stated), $\parallel$ is ordered concatenation, and $\llbracket Q \rrbracket$ is the bit indicating predicate Q . top/bot denote OFR subnetworks; l/r denote output directions. Subscripts cur/tmp denote current-layer/temporary objects. OSELECT(x, y, u) returns x for $u = 0$ and y for $u = 1$. Array lengths and traversal structure are public even when array contents are secret.

ALGORITHM INTERFACE DIRECTORY

Interfaces are grouped by the Supple sampler, the OFR network, and the OFRSupple sampler. Each signature, algorithm number, and page number links to the corresponding definition.

Interface	Purpose	Alg.	Page
Supple (compaction-based)			
SUPPLE (D, n, m, k):	Generates k shuffled samples of length m using membership marking and oblivious compaction.	1	3
SUPPLEFRONTIER (n, m, k):	Builds a public frontier of sample groups whose requested sizes fit within the input length.	2	3
SUPPLEMARK (n, m, k):	Marks membership for k independent uniform size- m samples drawn without replacement.	3	3
SUPPLECTR (M, F, n, m, k):	Counts, allocates, and fills the complete selection-bit array for the compaction-based sampler.	4	3
SUPPLEAPPLY (D, C, F, n, m, k):	Executes the stored selection bits from the start of the array and returns the shuffled samples.	5	3
SUPPLECTRCOUNT (F, n, m):	Counts the selection bits required by all compactions below the supplied frontier.	6	3
SUPPLECTRWRITE (M, C, F, n, m, p):	Writes selection bits in place while compacting and projecting membership for child nodes.	7	4
SUPPLECTRREAD (D, C, F, n, m, p):	Reads bits from offset p , recursively compacts data, and returns samples with the next offset.	8	4
OFR (Oblivious Fork Routing)			
OFRCOMPACT (D, T, n, n_l, n_r):	Generates controls and routes data in place; returns left/right views, duplicating shared records.	9	5
OFRCTR (T, n, n_l, n_r):	Normalizes routing tags, then allocates and fills the complete OFR control-word array.	10	5
OFRAPPLY (D, C, n, n_l, n_r):	Executes stored OFR controls in place and returns contiguous left/right views.	11	5
OFRCTRCOUNT (n, n_l, n_r):	Counts the two-bit control words required by the public input length and output capacities.	12	5
OFRCTRWRITE ($T, C, a, s, n, n_l, n_r, p$):	Routes normalized tags in place on a strided view, writes controls directly, and returns the next offset.	13	5
OFRCTRREAD ($D, C, a, s, n, n_l, n_r, p$):	Executes controls on a strided view of D in place and returns the next read offset.	14	5
OFRPAIRMASK (x, y):	Encodes an unordered pair of routing tags as a one-hot mask over the ten possible pair types.	15	6
OFRNORMALIZE (T, n, n_l, n_r):	Fills missing left and right routing responsibilities by relabeling zero tags only.	16	6
OFRBALANCE ($T, C, a, s, n, n_l, n_r, p$):	Writes one layer of controls directly, overwrites tag pairs in place, and returns the next offset.	17	6
OFORKCTR ($x, y, t^{\text{top}}, t^{\text{bot}}$):	Selects one OFork control word to realize the requested output routing responsibilities.	18	7
OFRSupple (OFR-based Supple)			
OFRSUPPLE (D, n, m, k):	Generates k shuffled samples of length m by combining compaction and OFR node allocation.	19	8
OFRSUPPLECTR (M, F, n, m, k):	Counts, allocates, and fills both the selection-bit and OFR control-word arrays for the sampler.	20	8
OFRSUPPLEAPPLY ($D, C_{\text{sw}}, C_{\text{ofr}}, F, n, m, k$):	Executes both control arrays from their starting positions and returns the shuffled samples.	21	8
OFRSUPPLECTRCOUNT (F, n, m, k):	Counts selection bits and OFR control words separately along the public sampling recursion.	22	8
OFRSUPPLECTRWRITE ($M, C_{\text{sw}}, C_{\text{ofr}}, F, n, m, k, p_{\text{sw}}, p_{\text{ofr}}$):	Writes both control streams while replaying compaction and OFR routing on membership words.	23	8
OFRSUPPLECTRREAD ($D, C_{\text{sw}}, C_{\text{ofr}}, F, n, m, k, p_{\text{sw}}, p_{\text{ofr}}$):	Consumes both control streams recursively and returns samples with both updated read offsets.	24	9

OFRSUPPLE reuses SUPPLEMARK (Algorithm 3), SUPPLEFRONTIER (Algorithm 2), and the OFR interfaces above. OCOMPACT, OSHUFFLE, OSELECT, and OFORK are assumed primitives without separate algorithm definitions.

Algorithm 1 SUPPLE(D, n, m, k)

Input: Secret data array D of public length n , public sample length m ($m < n$) and public sample count $k > 1$.

Output: Sample list $\mathcal{S} = [S_0, \dots, S_{k-1}]$, where $|S_j| = m$.

Offline phase

```
1:  $M \leftarrow \text{SUPPLEMARK}(n, m, k)$ 
2: if  $m \cdot k > n$  then
3:    $F \leftarrow \text{SUPPLEFRONTIER}(n, m, k)$ 
4: else
5:    $F \leftarrow \perp$ 
6: end if
7:  $C \leftarrow \text{SUPPECTR}(M, F, n, m, k)$ 
```

Online phase

```
8:  $\mathcal{S} \leftarrow \text{SUPPLEAPPLY}(D, C, F, n, m, k)$ 
9: return  $\mathcal{S}$ 
```

Algorithm 2 SUPPLEFRONTIER(n, m, k)

Input: Public input length n , public sample length m , and public sample count k .

Output: Public frontier array $F = [(s_0, k_0), \dots, (s_{r-1}, k_{r-1})]$ such that $m \cdot k_v \leq n$ for every node, $|k_u - k_v| \leq 1$ for any u, v , and r is minimum.

```
1:  $q \leftarrow \lfloor n/m \rfloor$   $\triangleright$  Maximum number of samples per frontier node
2:  $r \leftarrow \lceil k/q \rceil$   $\triangleright$  Minimum number of frontier nodes
3:  $b \leftarrow \lfloor k/r \rfloor$ 
4:  $t \leftarrow k \bmod r$   $\triangleright$  First  $t$  nodes contain one additional sample
5: Create an empty frontier array  $F$ 
6:  $s \leftarrow 0$ 
7: for  $v \leftarrow 0 \dots r-1$  do
8:   if  $v < t$  then
9:      $k_v \leftarrow b + 1$ 
10:  else
11:     $k_v \leftarrow b$ 
12:  end if
13:  Append  $(s, k_v)$  to  $F$ 
14:   $s \leftarrow s + k_v$ 
15: end for
16: return  $F$ 
```

Algorithm 3 SUPPLEMARK(n, m, k)

Input: Public data array length n , public sample length m and public sample count k .

Output: Secret membership-word array M of length n , where each $M[i]$ contains k bits.

```
1: Create a length- $n$  array  $M$  initialized to 0.
2: Create a length- $k$  array  $R$  initialized to  $m$ .
3: for  $i \leftarrow 0 \dots n-1$  do
4:   for  $j \leftarrow 0 \dots k-1$  do
5:     Generate  $\rho \in [0, 1)$  uniformly at random.
6:      $u \leftarrow \lfloor \rho < R[j]/(n-i) \rfloor$ 
```

```
7:    $M[i] \leftarrow M[i] \mid (u \ll j)$ 
8:    $R[j] \leftarrow R[j] - u$ 
9: end for
10: end for
11: return  $M$ 
```

Algorithm 4 SUPPECTR(M, F, n, m, k)

Input: Secret membership-word array M of public length n , public frontier array F , public sample length m and public sample count $k > 1$.

Output: Secret one-bit control array C .

```
1: if  $F \neq \perp$  then
2:    $F' \leftarrow F$ 
3: else
4:    $k_l \leftarrow \lfloor k/2 \rfloor$ ,  $k_r \leftarrow k - k_l$ 
5:    $F' \leftarrow [(0, k_l), (k_l, k_r)]$ 
6: end if
7:  $\ell_c \leftarrow \text{SUPPECTRCOUNT}(F', n, m)$ 
8: Create a length- $\ell_c$  bit array  $C$ .
9:  $p \leftarrow \text{SUPPECTRWRITE}(M, C, F', n, m, 0)$ 
10: return  $C$ 
```

Algorithm 5 SUPPLEAPPLY(D, C, F, n, m, k)

Input: Secret length- n data array D of public length n , secret one-bit control array C , public frontier array F , public sample length m and public sample count $k > 1$.

Output: Sample list $\mathcal{S} = [S_0, \dots, S_{k-1}]$, where $|S_j| = m$.

```
1: if  $F \neq \perp$  then
2:    $F' \leftarrow F$ 
3: else
4:    $k_l \leftarrow \lfloor k/2 \rfloor$ ,  $k_r \leftarrow k - k_l$ 
5:    $F' \leftarrow [(0, k_l), (k_l, k_r)]$ 
6: end if
7:  $(\mathcal{S}, p) \leftarrow \text{SUPPECTRREAD}(D, C, F', n, m, 0)$ 
8: return  $\mathcal{S}$ 
```

Algorithm 6 SUPPECTRCOUNT(F, n, m)

Input: Public frontier array F , current input length n and public sample length m .

Output: Number ℓ_c of one-bit control elements.

```
1:  $\ell_c \leftarrow 0$ 
2: for  $v \leftarrow 0 \dots |F| - 1$  do
3:    $(s_v, k_v) \leftarrow F[v]$ 
4:    $n_v \leftarrow \min(n, m \cdot k_v)$ 
5:   if  $n_v < n$  then
6:      $\ell_c \leftarrow \ell_c + n$ 
7:   end if
8:   if  $k_v > 1$  then
9:      $k_l \leftarrow \lfloor k_v/2 \rfloor$ ,  $k_r \leftarrow k_v - k_l$ 
10:     $F_v \leftarrow [(0, k_l), (k_l, k_r)]$ 
11:     $\ell_c \leftarrow \ell_c + \text{SUPPECTRCOUNT}(F_v, n_v, m)$ 
12:   end if
13: end for
```

14: **return** ℓ_c

Algorithm 7 SUPPLECTRWRITE(M, C, F, n, m, p)

Input: Secret membership-word array M of public length n , preallocated secret one-bit control array C , public frontier array F , public sample length m and write position p .

Output: Next free position p .

```
1: for  $v \leftarrow 0 \dots |F| - 1$  do
2:    $(s_v, k_v) \leftarrow F[v]$ 
3:    $n_v \leftarrow \min(n, m \cdot k_v)$ 
4:    $\mu_v \leftarrow (1 \ll k_v) - 1$ 
5:   Copy  $M$  and store it in  $M_{\text{tmp}}$ .
6:   if  $n_v < n$  then
7:     Create a length- $n$  bit array  $C_v$ .
8:     for  $i \leftarrow 0 \dots n - 1$  do
9:        $C_v[i] \leftarrow \llbracket ((M[i] \gg s_v) \& \mu_v) \neq 0 \rrbracket$ 
10:    end for
11:     $C[p : p + n] \leftarrow C_v$ 
12:     $p \leftarrow p + n$ 
13:    OCOMPACT( $M_{\text{tmp}}, C_v$ )
14:  end if
15:  Create a length- $n_v$  membership-word array  $M_v$ .
16:  for  $i \leftarrow 0 \dots n_v - 1$  do
17:     $M_v[i] \leftarrow (M_{\text{tmp}}[i] \gg s_v) \& \mu_v$ 
18:  end for
19:  if  $k_v > 1$  then
20:     $k_l \leftarrow \lfloor k_v/2 \rfloor, k_r \leftarrow k_v - k_l$ 
21:     $F_v \leftarrow [(0, k_l), (k_l, k_r)]$ 
22:     $p \leftarrow$ 
      SUPPLECTRWRITE( $M_v, C, F_v, n_v, m, p$ )
23:  end if
24: end for
25: return  $p$ 
```

Algorithm 8 SUPPLECTRREAD(D, C, F, n, m, p)

Input: Secret data array D of public length n , secret one-bit control array C , public frontier array F , public sample length m and read position p .

Output: Sample list $\mathcal{S}$ generated by the current subtree and next unread position p .

```
1: Create an empty sample list  $\mathcal{S}$ .
2: for  $v \leftarrow 0 \dots |F| - 1$  do
3:    $(s_v, k_v) \leftarrow F[v]$ 
4:    $n_v \leftarrow \min(n, m \cdot k_v)$ 
5:   Copy  $D$  and store it in  $D_{\text{tmp}}$ .
6:   if  $n_v < n$  then
7:      $C_v \leftarrow C[p : p + n]$ 
8:      $p \leftarrow p + n$ 
9:     OCOMPACT( $D_{\text{tmp}}, C_v$ )
10:  end if
11:   $D_v \leftarrow D_{\text{tmp}}[0 : n_v]$ 
12:  if  $k_v = 1$  then
13:     $S_v \leftarrow D_v[0 : m]$ 
```

```
14:   OSHUFFLE( $S_v$ )
15:    $\mathcal{S}_v \leftarrow [S_v]$ 
16: else
17:    $k_l \leftarrow \lfloor k_v/2 \rfloor, k_r \leftarrow k_v - k_l$ 
18:    $F_v \leftarrow [(0, k_l), (k_l, k_r)]$ 
19:    $(\mathcal{S}_v, p) \leftarrow$ 
     SUPPLECTRREAD( $D_v, C, F_v, n_v, m, p$ )
20: end if
21:    $\mathcal{S} \leftarrow \mathcal{S} \parallel \mathcal{S}_v$ 
22: end for
23: return  $(\mathcal{S}, p)$ 
```

Algorithm 9 OFRCompact(D, T, n, n_l, n_r)

Input: Mutable secret data array D and matching secret two-bit tag array T of public length n , public output capacities n_l, n_r satisfying $n_l + n_r = n$, $\sum_{i=0}^{n-1} T[i][0] \leq n_l$ and $\sum_{i=0}^{n-1} T[i][1] \leq n_r$.

Output: The first n_l slots of the mutated D form the left output and the remaining n_r slots form the right output; return both as views.

Offline phase

1: $C \leftarrow \text{OFRCTR}(T, n, n_l, n_r)$

Online phase

2: $(D_l, D_r) \leftarrow \text{OFRAPPLY}(D, C, n, n_l, n_r)$

3: **return** (D_l, D_r)

Algorithm 10 OFRCTR(T, n, n_l, n_r)

Input: Secret two-bit tag array T of public length n , public output capacities n_l, n_r satisfying $n_l + n_r = n$, $\sum_{i=0}^{n-1} T[i][0] \leq n_l$ and $\sum_{i=0}^{n-1} T[i][1] \leq n_r$.

Output: Secret two-bit control array C of length ℓ_c .

1: $\ell_c \leftarrow \text{OFRCTRCount}(n, n_l, n_r)$

2: Create array C of length ℓ_c .

3: $T' \leftarrow \text{OFRNORMALIZE}(T, n, n_l, n_r)$

4: $p \leftarrow \text{OFRCTRWRITE}(T', C, 0, 1, n, n_l, n_r, 0)$

5: **return** C

Algorithm 11 OFRAPPLY(D, C, n, n_l, n_r)

Input: Mutable secret data array D of public length n , secret two-bit control array C , public output capacities n_l, n_r satisfying $n_l + n_r = n$.

Output: Secret left and right views D_l, D_r of lengths n_l, n_r into the mutated backing array D .

1: $p \leftarrow \text{OFRCTRREAD}(D, C, 0, 1, n, n_l, n_r, 0)$

2: $D_l \leftarrow D[0 : n_l]$, $D_r \leftarrow D[n_l : n] \triangleright$ Views; no data copy

3: **return** (D_l, D_r)

Algorithm 12 OFRCTRCount(n, n_l, n_r)

Input: Public input length n , public output capacities n_l, n_r satisfying $n_l + n_r = n$.

Output: Number ℓ_c of two-bit OFORK control words.

1: **if** $n_l = 0$ or $n_r = 0$ **then**

2: **return** 0

3: **end if**

4: **if** $n = 2$ **then**

5: **return** 1

6: **end if**

7: $\ell_g \leftarrow \lfloor n/2 \rfloor$

8: $n^{\text{top}} \leftarrow \lceil n/2 \rceil$

9: $n_l^{\text{top}} \leftarrow \lceil n_l/2 \rceil$, $n_r^{\text{top}} \leftarrow n^{\text{top}} - n_l^{\text{top}}$

10: $n_l^{\text{bot}} \leftarrow n_l - n_l^{\text{top}}$, $n_r^{\text{bot}} \leftarrow n_r - n_r^{\text{top}}$

11: $\ell_c^{\text{top}} \leftarrow \text{OFRCTRCount}(n^{\text{top}}, n_l^{\text{top}}, n_r^{\text{top}})$

12: $\ell_c^{\text{bot}} \leftarrow \text{OFRCTRCount}(\ell_g, n_l^{\text{bot}}, n_r^{\text{bot}})$

13: **return** $\ell_g + \ell_c^{\text{top}} + \ell_c^{\text{bot}}$

Algorithm 13 OFRCTRWRITE($T, C, a, s, n, n_l, n_r, p$)

Input: Mutable secret tag buffer T and preallocated secret control array C ; public base $a \geq 0$, stride $s > 0$, length $n \geq 1$, capacities $n_l + n_r = n$, and cursor p , with $a + (n-1)s < |T|$. The normalized logical tags $T[a + js]$ have exact responsibilities n_l, n_r .

Output: Write controls from $C[p]$, overwrite the current tag view, and return the next cursor. Tags outside the view are unchanged.

1: **if** $n_l = 0$ or $n_r = 0$ **then**

2: **return** $p \triangleright$ All inputs go to one side, no control word

3: **end if**

4: **if** $n = 2$ **then**

5: $C[p] \leftarrow \text{OFORKCTR}(T[a], T[a + s], 10, 01)$

6: $(T[a], T[a + s]) \leftarrow (10, 01)$

7: **return** $p + 1$

8: **end if**

9: $n^{\text{top}} \leftarrow \lceil n/2 \rceil$, $n^{\text{bot}} \leftarrow n - n^{\text{top}}$

10: $n_l^{\text{top}} \leftarrow \lceil n_l/2 \rceil$, $n_r^{\text{top}} \leftarrow n^{\text{top}} - n_l^{\text{top}}$

11: $n_l^{\text{bot}} \leftarrow n_l - n_l^{\text{top}}$, $n_r^{\text{bot}} \leftarrow n_r - n_r^{\text{top}}$

12: $p \leftarrow \text{OFRBALANCE}(T, C, a, s, n, n_l, n_r, p)$

13: $p \leftarrow \text{OFRCTRWRITE}(T, C, a, 2s, n^{\text{top}},$
 $n_l^{\text{top}}, n_r^{\text{top}}, p)$

14: $p \leftarrow \text{OFRCTRWRITE}(T, C, a + s, 2s, n^{\text{bot}},$
 $n_l^{\text{bot}}, n_r^{\text{bot}}, p)$

15: **return** p

Algorithm 14 OFRCTRREAD($D, C, a, s, n, n_l, n_r, p$)

Input: Mutable secret backing array D ; the current length- n logical subnetwork occupies $D[a + js]$ for $j = 0, \dots, n-1$, with public base $a \geq 0$ and stride $s > 0$ satisfying $a + (n-1)s < |D|$, secret two-bit control array C , public output capacities n_l, n_r satisfying $n_l + n_r = n$, read position p .

Output: The first n_l logical positions contain the left output and the remaining n_r positions contain the right output in the same backing array; return the next unread position p in C .

1: **if** $n_l = 0$ or $n_r = 0$ **then**

2: **return** p

3: **end if**

4: **if** $n = 2$ **then**

5: $(x, y) \leftarrow (D[a], D[a + s])$

6: $(D[a], D[a + s]) \leftarrow \text{OFORK}(x, y, C[p])$ $\triangleright$

Simultaneous write after reading both inputs

7: **return** $p + 1$

8: **end if**

9: $\ell_g \leftarrow \lfloor n/2 \rfloor$

10: $n^{\text{top}} \leftarrow \lceil n/2 \rceil$, $n^{\text{bot}} \leftarrow n - n^{\text{top}}$

11: $n_l^{\text{top}} \leftarrow \lceil n_l/2 \rceil$, $n_r^{\text{top}} \leftarrow n^{\text{top}} - n_l^{\text{top}}$

12: $n_l^{\text{bot}} \leftarrow n_l - n_l^{\text{top}}$, $n_r^{\text{bot}} \leftarrow n_r - n_r^{\text{top}}$

13: **for** $i \leftarrow 0 \dots \ell_g - 1$ **do**

14: $j_0 \leftarrow a + 2is$, $j_1 \leftarrow j_0 + s$

15: $(x, y) \leftarrow (D[j_0], D[j_1])$

```

16:  $(D[j_0], D[j_1]) \leftarrow \text{OFORK}(x, y, C[p + i])$ 
17: end for
    If  $n$  is odd, the last logical wire bypasses to the top view.
18:  $p \leftarrow p + \ell_g$ 
19:  $p \leftarrow \text{OFRCTRREAD}(D, C, a, 2s, n^{\text{top}}, n_l^{\text{top}}, n_r^{\text{top}}, p)$ 
20:  $p \leftarrow \text{OFRCTRREAD}(D, C, a + s, 2s, n^{\text{bot}}, n_l^{\text{bot}}, n_r^{\text{bot}}, p)$ 
21: return  $p$ 

```

Algorithm 15 OFRPAIRMASK(x, y)

Input: Two secret two-bit routing tags x and y .

Output: A secret one-hot array $P \in \{0, 1\}^{10}$ for the unordered two-bit tag types $\{00, 00\}$, $\{00, 10\}$, $\{00, 01\}$, $\{00, 11\}$, $\{10, 10\}$, $\{10, 01\}$, $\{10, 11\}$, $\{01, 01\}$, $\{01, 11\}$, and $\{11, 11\}$, in this order.

```

1:  $P[0] \leftarrow \llbracket x = 00 \rrbracket \wedge \llbracket y = 00 \rrbracket$ 
2:  $P[1] \leftarrow (\llbracket x = 00 \rrbracket \wedge \llbracket y = 10 \rrbracket) \vee (\llbracket x = 10 \rrbracket \wedge \llbracket y = 00 \rrbracket)$ 
3:  $P[2] \leftarrow (\llbracket x = 00 \rrbracket \wedge \llbracket y = 01 \rrbracket) \vee (\llbracket x = 01 \rrbracket \wedge \llbracket y = 00 \rrbracket)$ 
4:  $P[3] \leftarrow (\llbracket x = 00 \rrbracket \wedge \llbracket y = 11 \rrbracket) \vee (\llbracket x = 11 \rrbracket \wedge \llbracket y = 00 \rrbracket)$ 
5:  $P[4] \leftarrow \llbracket x = 10 \rrbracket \wedge \llbracket y = 10 \rrbracket$ 
6:  $P[5] \leftarrow (\llbracket x = 10 \rrbracket \wedge \llbracket y = 01 \rrbracket) \vee (\llbracket x = 01 \rrbracket \wedge \llbracket y = 10 \rrbracket)$ 
7:  $P[6] \leftarrow (\llbracket x = 10 \rrbracket \wedge \llbracket y = 11 \rrbracket) \vee (\llbracket x = 11 \rrbracket \wedge \llbracket y = 10 \rrbracket)$ 
8:  $P[7] \leftarrow \llbracket x = 01 \rrbracket \wedge \llbracket y = 01 \rrbracket$ 
9:  $P[8] \leftarrow (\llbracket x = 01 \rrbracket \wedge \llbracket y = 11 \rrbracket) \vee (\llbracket x = 11 \rrbracket \wedge \llbracket y = 01 \rrbracket)$ 
10:  $P[9] \leftarrow \llbracket x = 11 \rrbracket \wedge \llbracket y = 11 \rrbracket$ 
11: return  $P$ 

```

Algorithm 16 OFRNORMALIZE(T, n, n_l, n_r)

Input: Secret two-bit tag array T of public length n , public output capacities n_l, n_r satisfying $n_l + n_r = n$, $\sum_{i=0}^{n-1} T[i][0] \leq n_l$ and $\sum_{i=0}^{n-1} T[i][1] \leq n_r$.

Output: A normalized secret two-bit tag array T' satisfying $\sum_{i=0}^{n-1} T'[i][0] = n_l$ and $\sum_{i=0}^{n-1} T'[i][1] = n_r$, and every nonzero input tag remains unchanged.

```

1:  $w_l \leftarrow 0, w_r \leftarrow 0$   $\triangleright$  Current left and right responsibilities
2: for  $i \leftarrow 0 \dots n - 1$  do
3:    $w_l \leftarrow w_l + T[i][0]$ 
4:    $w_r \leftarrow w_r + T[i][1]$ 
5: end for
6:  $\delta_l \leftarrow n_l - w_l$   $\triangleright$  Number of 00 tags assigned to the left
7:  $\delta_r \leftarrow n_r - w_r$   $\triangleright$  Number of 00 tags assigned to the right
8: Create an array  $T'$  of length  $n$ .
9:  $r_0 \leftarrow 0$   $\triangleright$  Number of preceding 00 tags
10: for  $i \leftarrow 0 \dots n - 1$  do
11:    $z \leftarrow \llbracket T[i] = 00 \rrbracket$ 
12:    $u_l \leftarrow z \wedge \llbracket r_0 < \delta_l \rrbracket$ 
13:    $u_r \leftarrow z \wedge \llbracket \delta_l \leq r_0 \rrbracket \wedge \llbracket r_0 < \delta_l + \delta_r \rrbracket$ 
14:    $T'[i] \leftarrow T[i]$ 
15:    $T'[i] \leftarrow \text{OSELECT}(T'[i], 10, u_l)$ 
16:    $T'[i] \leftarrow \text{OSELECT}(T'[i], 01, u_r)$ 
17:    $r_0 \leftarrow r_0 + z$ 

```

```

18: end for
19: return  $T'$ 

```

Algorithm 17 OFRBALANCE($T, C, a, s, n, n_l, n_r, p$)

Input: Mutable secret tag buffer T , preallocated secret control array C , public $a \geq 0, s > 0, n \geq 3$, capacities $n_l + n_r = n$, and cursor p . Logical tags $T[a + js]$ have exact responsibilities n_l, n_r ; $a + (n - 1)s < |T|$.

Output: Write $\lfloor n/2 \rfloor$ controls directly into C and overwrite each tag pair with its top/bottom output tags; return the next cursor. The top/bottom views have bases $a, a + s$ and stride $2s$.

```

1:  $\ell_g \leftarrow \lfloor n/2 \rfloor$ 
2:  $n^{\text{top}} \leftarrow \lceil n/2 \rceil$ 
3:  $n_l^{\text{top}} \leftarrow \lceil n_l/2 \rceil, n_r^{\text{top}} \leftarrow n^{\text{top}} - n_l^{\text{top}}$ 
4:  $e_l \leftarrow 0, e_r \leftarrow 0$ 
5: if  $n \bmod 2 = 1$  then
6:    $e_l \leftarrow T[a + (n - 1)s][0], e_r \leftarrow T[a + (n - 1)s][1]$   $\triangleright$ 
   The last input bypasses to the top
7: end if
8:  $n_l^{\text{top}'} \leftarrow n_l^{\text{top}} - e_l, n_r^{\text{top}'} \leftarrow n_r^{\text{top}} - e_r$   $\triangleright$  Secret
   paired-input responsibility targets
9: Create array  $H$  of length 10, initialized to 0  $\triangleright$  Tag-pair
   type counts
10: for  $i \leftarrow 0 \dots \ell_g - 1$  do
11:    $j_0 \leftarrow a + 2is, j_1 \leftarrow j_0 + s$ 
12:    $(x, y) \leftarrow (T[j_0], T[j_1])$ 
13:    $P \leftarrow \text{OFRPAIRMASK}(x, y)$ 
14:   for  $j \leftarrow 0 \dots 9$  do
15:      $H[j] \leftarrow H[j] + P[j]$ 
16:   end for
17: end for
18:  $d \leftarrow H[5]$   $\triangleright$  Number of  $\{10, 01\}$  pairs
19:  $v_l \leftarrow H[1] + H[3] + H[8]$ 
20:  $v_r \leftarrow H[2] + H[3] + H[6]$ 
21:  $f_l \leftarrow H[4] + H[6] + H[9]$ 
22:  $f_r \leftarrow H[7] + H[8] + H[9]$   $\triangleright$  Fixed top responsibilities
23:  $q_d^{\text{min}} \leftarrow \max\{0, n_l^{\text{top}'} - f_l - v_l, f_r + d - n_r^{\text{top}'}\}$ 
24:  $q_d^{\text{max}} \leftarrow \min\{d, n_l^{\text{top}'} - f_l, f_r + d + v_r - n_r^{\text{top}'}\}$ 
25:  $q_d \leftarrow \min\{q_d^{\text{max}}, \max\{q_d^{\text{min}}, \lfloor d/2 \rfloor\}\}$   $\triangleright$  The interval is
   nonempty
26:  $q_l \leftarrow n_l^{\text{top}'} - f_l - q_d$ 
27:  $q_r \leftarrow n_r^{\text{top}'} - f_r - d + q_d$ 
28:  $r_d \leftarrow 0, r_l \leftarrow 0, r_r \leftarrow 0$ 
29: for  $i \leftarrow 0 \dots \ell_g - 1$  do
30:    $j_0 \leftarrow a + 2is, j_1 \leftarrow j_0 + s$ 
31:    $(x, y) \leftarrow (T[j_0], T[j_1])$ 
32:    $P \leftarrow \text{OFRPAIRMASK}(x, y)$ 
33:    $u_d \leftarrow P[5] \wedge \llbracket r_d < q_d \rrbracket$ 
34:    $u_l \leftarrow (P[1] \vee P[3] \vee P[8]) \wedge \llbracket r_l < q_l \rrbracket$ 
35:    $u_r \leftarrow (P[2] \vee P[3] \vee P[6]) \wedge \llbracket r_r < q_r \rrbracket$ 
36:    $z_l \leftarrow P[4] \vee P[6] \vee P[9]$ 
37:    $z_r \leftarrow P[7] \vee P[8] \vee P[9]$ 
38:    $t_l^{\text{top}} \leftarrow z_l \vee u_d \vee u_l$ 

```

```

39:   $t_r^{\text{top}} \leftarrow z_r \vee (P[5] \wedge \neg u_d) \vee u_r$ 
40:   $t_l^{\text{bot}} \leftarrow x[0] + y[0] - t_l^{\text{top}}$ 
41:   $t_r^{\text{bot}} \leftarrow x[1] + y[1] - t_r^{\text{top}}$ 
42:   $t^{\text{top}} \leftarrow (t_l^{\text{top}}, t_r^{\text{top}})$ 
43:   $t^{\text{bot}} \leftarrow (t_l^{\text{bot}}, t_r^{\text{bot}})$ 
44:   $C[p+i] \leftarrow \text{OFORKCTR}(x, y, t^{\text{top}}, t^{\text{bot}})$ 
45:   $(T[j_0], T[j_1]) \leftarrow (t^{\text{top}}, t^{\text{bot}}) \triangleright$  Both inputs were read
    before either write
46:   $r_d \leftarrow r_d + P[5]$ 
47:   $r_l \leftarrow r_l + (P[1] \vee P[3] \vee P[8])$ 
48:   $r_r \leftarrow r_r + (P[2] \vee P[3] \vee P[6])$ 
49: end for
    An odd last tag stays in place in the top view.
50: return  $p + \ell_g$ 

```

Algorithm 18 $\text{OFORKCTR}(x, y, t^{\text{top}}, t^{\text{bot}})$

Input: Secret input two-bit tags x, y and desired secret output two-bit tags $t^{\text{top}}, t^{\text{bot}}$.

Output: A secret two-bit OFORK control word c .

```

1:  $z_{\text{straight}} \leftarrow \llbracket t^{\text{top}} = x \rrbracket \wedge \llbracket t^{\text{bot}} = y \rrbracket$ 
2:  $z_{\text{cross}} \leftarrow \llbracket t^{\text{top}} = y \rrbracket \wedge \llbracket t^{\text{bot}} = x \rrbracket$ 
3:  $z_x \leftarrow \llbracket x = 11 \rrbracket \wedge \llbracket y = 00 \rrbracket$ 
4:  $z_y \leftarrow \llbracket x = 00 \rrbracket \wedge \llbracket y = 11 \rrbracket$ 
5:  $z_{\text{copy}} \leftarrow z_x \vee z_y$ 
6:  $z_{l,r} \leftarrow \llbracket t^{\text{top}} = 10 \rrbracket \wedge \llbracket t^{\text{bot}} = 01 \rrbracket$ 
7:  $z_{r,l} \leftarrow \llbracket t^{\text{top}} = 01 \rrbracket \wedge \llbracket t^{\text{bot}} = 10 \rrbracket$ 
8:  $z_{\text{fork}} \leftarrow z_{\text{copy}} \wedge (z_{l,r} \vee z_{r,l})$ 
9:  $c_{\text{fork}} \leftarrow (z_y, z_y) \triangleright$  00 copies  $x$ ; 11 copies  $y$ 
10:  $u_{\text{cross}} \leftarrow z_{\text{cross}} \wedge \neg z_{\text{straight}}$ 
11:  $u_{\text{fork}} \leftarrow z_{\text{fork}} \wedge \neg z_{\text{straight}} \wedge \neg z_{\text{cross}}$ 
12:  $c \leftarrow$  01
13:  $c \leftarrow \text{OSELECT}(c, \mathbf{10}, u_{\text{cross}})$ 
14:  $c \leftarrow \text{OSELECT}(c, c_{\text{fork}}, u_{\text{fork}})$ 
15: return  $c$ 

```

Algorithm 19 OFRSUPPLE(D, n, m, k)

Input: Mutable secret data array D of public length n , public sample length m ($1 \leq m < n$) and public sample count $k > 1$.

Output: Sample list $\mathcal{S} = [S_0, \dots, S_{k-1}]$, where $|S_j| = m$.

Offline phase

```
1:  $M \leftarrow \text{SUPPLEMARK}(n, m, k)$ 
2: if  $m \cdot k > n$  then
3:    $F \leftarrow \text{SUPPLEFRONTIER}(n, m, k)$ 
4: else
5:    $F \leftarrow \perp$ 
6: end if
7:  $(C_{\text{sw}}, C_{\text{ofr}}) \leftarrow \text{OFRSUPPLECTR}(M, F, n, m, k)$ 
Online phase
8:  $\mathcal{S} \leftarrow \text{OFRSUPPLEAPPLY}(D, C_{\text{sw}}, C_{\text{ofr}}, F, n, m, k)$ 
9: return  $\mathcal{S}$ 
```

Algorithm 20 OFRSUPPLECTR(M, F, n, m, k)

Input: Secret k -bit membership-word array M of public length n , public frontier array F or sentinel $\perp$, public sample length m and public sample count $k > 1$. Each sample bit is set at exactly m positions.

Output: Secret one-bit control array C_{sw} and secret two-bit control array C_{ofr} .

```
1:  $(\ell_{c,\text{sw}}, \ell_{c,\text{ofr}}) \leftarrow \text{OFRSUPPLECTRCOUNT}(F, n, m, k)$ 
2: Create a length- $\ell_{c,\text{sw}}$  one-bit control array  $C_{\text{sw}}$ .
3: Create a length- $\ell_{c,\text{ofr}}$  two-bit control array  $C_{\text{ofr}}$ .
4:  $(p_{\text{sw}}, p_{\text{ofr}}) \leftarrow \text{OFRSUPPLECTRWRITE}(M, C_{\text{sw}}, C_{\text{ofr}}, F, n, m, k, 0, 0)$ 
5: return  $(C_{\text{sw}}, C_{\text{ofr}})$ 
```

Algorithm 21 OFRSUPPLEAPPLY($D, C_{\text{sw}}, C_{\text{ofr}}, F, n, m, k$)

Input: Mutable secret data array D of public length n , matching secret one-bit and two-bit control arrays $C_{\text{sw}}, C_{\text{ofr}}$, public frontier array F or sentinel $\perp$, public sample length m and public sample count $k > 1$.

Output: Sample list $\mathcal{S} = [S_0, \dots, S_{k-1}]$, where $|S_j| = m$.

```
1:  $(\mathcal{S}, p_{\text{sw}}, p_{\text{ofr}}) \leftarrow \text{OFRSUPPLECTRREAD}(D, C_{\text{sw}}, C_{\text{ofr}}, F, n, m, k, 0, 0)$ 
2: return  $\mathcal{S}$ 
```

Algorithm 22 OFRSUPPLECTRCOUNT(F, n, m, k)

Input: Public frontier array F or sentinel $\perp$, public input length n , public sample length m and public sample count $k \geq 1$. For $k = 1$, $n = m$.

Output: Counts $(\ell_{c,\text{sw}}, \ell_{c,\text{ofr}})$, in bits and two-bit words, respectively

```
1: if  $k = 1$  then
2:   return  $(0, 0)$ 
3: end if
4:  $k_l \leftarrow \lfloor k/2 \rfloor$ ,  $k_r \leftarrow k - k_l$ 
```

```
5:  $(\ell_{c,\text{sw}}, \ell_{c,\text{ofr}}) \leftarrow (0, 0)$ 
6: if  $n = m \cdot k$  then
7:    $n_l \leftarrow m \cdot k_l$ ,  $n_r \leftarrow m \cdot k_r$ 
8:    $\ell_{c,\text{ofr}} \leftarrow \text{OFRCTRCount}(n, n_l, n_r)$ 
9:    $F' \leftarrow [(0, k_l), (k_l, k_r)]$ 
10: else if  $F \neq \perp$  then
11:    $F' \leftarrow F$ 
12: else
13:    $F' \leftarrow [(0, k_l), (k_l, k_r)]$ 
14: end if
15: for  $v \leftarrow 0 \dots |F'| - 1$  do
16:    $(s_v, k_v) \leftarrow F'[v]$ 
17:    $n_v \leftarrow \min(n, m \cdot k_v)$ 
18:   if  $n \neq m \cdot k$  and  $n_v < n$  then
19:      $\ell_{c,\text{sw}} \leftarrow \ell_{c,\text{sw}} + n$ 
20:   end if
21:  $(\ell'_{c,\text{sw}}, \ell'_{c,\text{ofr}}) \leftarrow \text{OFRSUPPLECTRCOUNT}(\perp, n_v, m, k_v)$ 
22:    $\ell_{c,\text{sw}} \leftarrow \ell_{c,\text{sw}} + \ell'_{c,\text{sw}}$ 
23:    $\ell_{c,\text{ofr}} \leftarrow \ell_{c,\text{ofr}} + \ell'_{c,\text{ofr}}$ 
24: end for
25: return  $(\ell_{c,\text{sw}}, \ell_{c,\text{ofr}})$ 
```

Algorithm 23 OFRSUPPLECTRWRITE($M, C_{\text{sw}}, C_{\text{ofr}}, F, n, m, k, p_{\text{sw}}, p_{\text{ofr}}$)

Input: Mutable secret k -bit membership-word array M of public length n , preallocated secret one-bit and two-bit control arrays $C_{\text{sw}}, C_{\text{ofr}}$, public frontier array F or sentinel $\perp$, public m, k , and public write positions $p_{\text{sw}}, p_{\text{ofr}}$. Each sample bit is set at exactly m positions; $k = 1$ requires $n = m$.

Output: Next positions $(p_{\text{sw}}, p_{\text{ofr}})$ after filling both arrays in place

```
1: if  $k = 1$  then
2:   return  $(p_{\text{sw}}, p_{\text{ofr}})$ 
3: end if
4:  $k_l \leftarrow \lfloor k/2 \rfloor$ ,  $k_r \leftarrow k - k_l$ 
5: if  $n = m \cdot k$  then
6:    $n_l \leftarrow m \cdot k_l$ ,  $n_r \leftarrow m \cdot k_r$ 
7:    $\mu_l \leftarrow (1 \ll k_l) - 1$ ,  $\mu_r \leftarrow (1 \ll k_r) - 1$ 
8:   Create a length- $n$  two-bit tag array  $T$ 
9:   for  $i \leftarrow 0 \dots n - 1$  do
10:     $T[i][0] \leftarrow \llbracket (M[i] \& \mu_l) \neq 0 \rrbracket$ 
11:     $T[i][1] \leftarrow \llbracket ((M[i] \gg k_l) \& \mu_r) \neq 0 \rrbracket$ 
12:   end for
13:    $T' \leftarrow \text{OFRNORMALIZE}(T, n, n_l, n_r)$ 
14:    $p_{\text{tmp}} \leftarrow p_{\text{ofr}}$ 
15:    $p_{\text{ofr}} \leftarrow \text{OFRCTRWRITE}(T', C_{\text{ofr}}, 0, 1, n, n_l, n_r, p_{\text{ofr}})$ 
16:    $p_{\text{tmp}} \leftarrow \text{OFRCTRREAD}(M, C_{\text{ofr}}, 0, 1, n, n_l, n_r, p_{\text{tmp}})$ 
17:    $M_l \leftarrow M[0 : n_l]$ ,  $M_r \leftarrow M[n_l : n]$   $\triangleright$  Disjoint views of the in-place replay
   Project membership after the complete OFR replay.
```

```

18: for  $i \leftarrow 0 \dots n_l - 1$  do
19:    $M_l[i] \leftarrow M_l[i] \& \mu_l$ 
20: end for
21: for  $i \leftarrow 0 \dots n_r - 1$  do
22:    $M_r[i] \leftarrow (M_r[i] \gg k_l) \& \mu_r$ 
23: end for
24:  $(p_{\text{swo}}, p_{\text{ofr}}) \leftarrow \text{OFRSUPPLECTRWRITE}(\$ 
25:    $M_l, C_{\text{swo}}, C_{\text{ofr}}, \perp, n_l, m, k_l, p_{\text{swo}}, p_{\text{ofr}})$ 
26:  $(p_{\text{swo}}, p_{\text{ofr}}) \leftarrow \text{OFRSUPPLECTRWRITE}(\$ 
27:    $M_r, C_{\text{swo}}, C_{\text{ofr}}, \perp, n_r, m, k_r, p_{\text{swo}}, p_{\text{ofr}})$ 
28: else
29:   if  $F \neq \perp$  then
30:      $F' \leftarrow F$ 
31:   else
32:      $F' \leftarrow [(0, k_l), (k_l, k_r)]$ 
33:   end if
34:   for  $v \leftarrow 0 \dots |F'| - 1$  do
35:      $(s_v, k_v) \leftarrow F'[v]$ 
36:      $n_v \leftarrow \min(n, m \cdot k_v)$ ,  $\mu_v \leftarrow (1 \ll k_v) - 1$ 
37:     Copy  $M$  and store it in  $M_{\text{tmp}}$ .
38:     if  $n_v < n$  then
39:       Create a length- $n$  bit array  $C_v$ 
40:       for  $i \leftarrow 0 \dots n - 1$  do
41:          $C_v[i] \leftarrow \llbracket (M[i] \gg s_v) \& \mu_v \rrbracket \neq 0$ 
42:       end for
43:        $C_{\text{swo}}[p_{\text{swo}} : p_{\text{swo}} + n] \leftarrow C_v$ 
44:        $p_{\text{swo}} \leftarrow p_{\text{swo}} + n$ 
45:        $\text{OCOMPACT}(M_{\text{tmp}}, C_v)$ 
46:     end if
47:     Create a length- $n_v$  membership-word array  $M_v$ .
48:     for  $i \leftarrow 0 \dots n_v - 1$  do
49:        $M_v[i] \leftarrow (M_{\text{tmp}}[i] \gg s_v) \& \mu_v$ 
50:     end for
51:      $(p_{\text{swo}}, p_{\text{ofr}}) \leftarrow \text{OFRSUPPLECTRWRITE}(\$ 
52:        $M_v, C_{\text{swo}}, C_{\text{ofr}}, \perp, n_v, m, k_v, p_{\text{swo}}, p_{\text{ofr}})$ 
53:   end if
54: end if
55: return  $(p_{\text{swo}}, p_{\text{ofr}})$ 

```

Algorithm 24 OFRSUPPLECTRWRITE($D, C_{\text{swo}}, C_{\text{ofr}}, F, n, m, k, p_{\text{swo}}, p_{\text{ofr}}$)

Input: Mutable secret data array D of public length n , matching secret one-bit and two-bit control arrays $C_{\text{swo}}, C_{\text{ofr}}$, public frontier array F or sentinel $\perp$, public m, k , and public read positions $p_{\text{swo}}, p_{\text{ofr}}$. For $k = 1$, $n = m$.

Output: Ordered samples $\mathcal{S}$ and next unread positions $(p_{\text{swo}}, p_{\text{ofr}})$

```

1: if  $k = 1$  then
2:   assert  $n = m$ 
3:   Copy  $D[0 : m)$  into  $\mathcal{S}$ 
4:    $\text{OSHUFFLE}(\mathcal{S})$ 
5:   return  $([\mathcal{S}], p_{\text{swo}}, p_{\text{ofr}})$ 
6: end if
7:  $k_l \leftarrow \lfloor k/2 \rfloor$ ,  $k_r \leftarrow k - k_l$ 

```

```

8: if  $n = m \cdot k$  then
9:    $n_l \leftarrow m \cdot k_l$ ,  $n_r \leftarrow m \cdot k_r$ 
10:   $p_{\text{ofr}} \leftarrow \text{OFRCTRREAD}(\$ 
11:     $D, C_{\text{ofr}}, 0, 1, n, n_l, n_r, p_{\text{ofr}})$ 
12:   $D_l \leftarrow D[0 : n_l)$ ,  $D_r \leftarrow D[n_l : n)$   $\triangleright$  Disjoint views;
    no data copy
13:   $(\mathcal{S}_l, p_{\text{swo}}, p_{\text{ofr}}) \leftarrow \text{OFRSUPPLECTRWRITE}(\$ 
14:     $D_l, C_{\text{swo}}, C_{\text{ofr}}, \perp, n_l, m, k_l, p_{\text{swo}}, p_{\text{ofr}})$ 
15:   $(\mathcal{S}_r, p_{\text{swo}}, p_{\text{ofr}}) \leftarrow \text{OFRSUPPLECTRWRITE}(\$ 
16:     $D_r, C_{\text{swo}}, C_{\text{ofr}}, \perp, n_r, m, k_r, p_{\text{swo}}, p_{\text{ofr}})$ 
17:  return  $(\mathcal{S}_l \parallel \mathcal{S}_r, p_{\text{swo}}, p_{\text{ofr}})$ 
18: end if
19: if  $F \neq \perp$  then
20:    $F' \leftarrow F$ 
21: else
22:    $F' \leftarrow [(0, k_l), (k_l, k_r)]$ 
23: end if
24:  $\mathcal{S} \leftarrow []$ 
25: for  $v \leftarrow 0 \dots |F'| - 1$  do
26:    $(s_v, k_v) \leftarrow F'[v]$ 
27:    $n_v \leftarrow \min(n, m \cdot k_v)$ 
28:   Copy  $D$  and store it in  $D_{\text{tmp}}$ .
29:   if  $n_v < n$  then
30:      $C_v \leftarrow C_{\text{swo}}[p_{\text{swo}} : p_{\text{swo}} + n)$ 
31:      $p_{\text{swo}} \leftarrow p_{\text{swo}} + n$ 
32:      $\text{OCOMPACT}(D_{\text{tmp}}, C_v)$ 
33:   end if
34:    $D_v \leftarrow D_{\text{tmp}}[0 : n_v)$ 
35:    $(\mathcal{S}_v, p_{\text{swo}}, p_{\text{ofr}}) \leftarrow \text{OFRSUPPLECTRWRITE}(\$ 
36:      $D_v, C_{\text{swo}}, C_{\text{ofr}}, \perp, n_v, m, k_v, p_{\text{swo}}, p_{\text{ofr}})$ 
37:    $\mathcal{S} \leftarrow \mathcal{S} \parallel \mathcal{S}_v$ 
38: end for
39: return  $(\mathcal{S}, p_{\text{swo}}, p_{\text{ofr}})$ 

```
